Docker & Kubernetes

Introdução

A infraestrutura atual da Web Solutions atingiu o limite de escala. O modelo legado baseado em **Máquinas Virtuais** é insustentável.

Cada nova aplicação provisionada carrega o peso morto de um sistema operacional visitante (guest OS) completo. Isso força a emulação de hardware e pulveriza recursos de CPU e RAM apenas para manter processos repetitivos rodando em background.

O resultado direto é redundância, lentidão nos deploys e uma alocação de recursos engessada.

A solução técnica para esse gargalo é a conteinerização com **Docker** orquestrada por **Kubernetes**. O Docker destrói a ineficiência das VMs descartando a virtualização de hardware. Ele aproveita o kernel do próprio servidor hospedeiro e isola as aplicações logicamente.

O overhead cai drasticamente. A aplicação roda nativamente, mas trancada na sua própria bolha de execução, consumindo apenas o que realmente precisa.

Este documento detalha a prova de conceito exigida para modernizar a arquitetura da empresa.

O objetivo é provisionar a coexistência isolada de dois servidores web distintos dentro de um único cluster Kubernetes:

- **Nginx** (porta **8080**)
- **Apache HTTPD** (porta **8081**)

O foco é abandonar a configuração manual baseada em scripts imperativos e adotar a **Infraestrutura como Código (IaC)**, garantindo um ambiente elástico, resiliente e executado com máxima eficiência de recursos.

Docker Official Documentation: https://docs.docker.com/ - Referência primária.

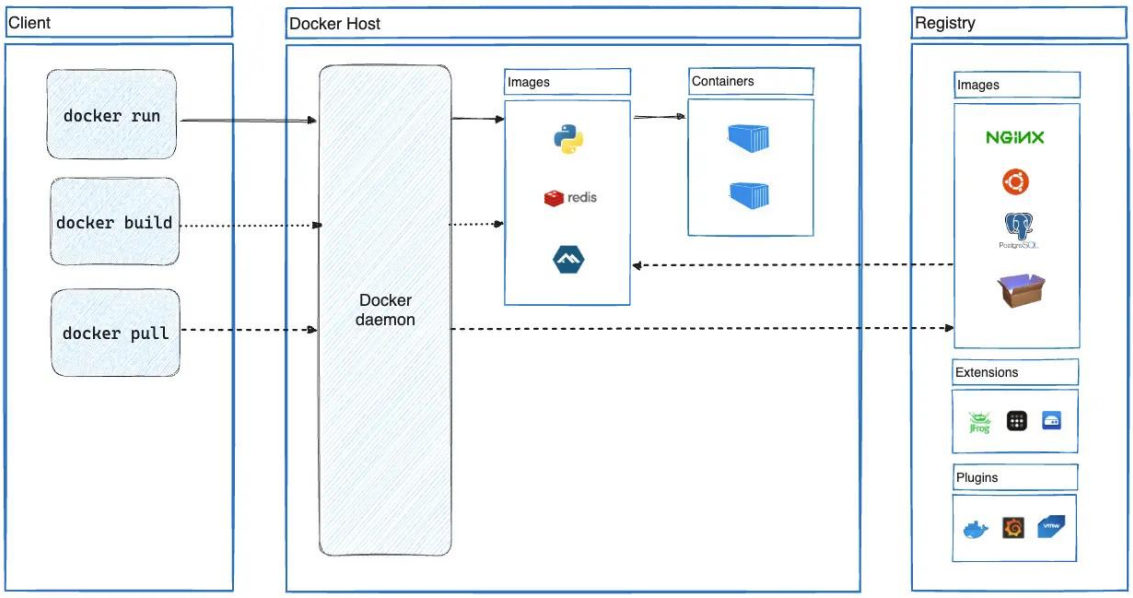
Kubernetes Official Documentation: https://kubernetes.io/docs/ - Base para o desenho dos manifestos YAML, ciclo de vida dos Pods e roteamento de rede via Services.

Estrutura do Docker

O problema histórico da engenharia de software sempre foi o abismo entre o ambiente de desenvolvimento e o servidor de produção. O sintoma clássico é a falácia do "na minha máquina funciona".

As dependências divergem, as bibliotecas conflitam e o *deploy* falha. A virtualização tradicional tentou sanar isso emulando hardware, mas ao custo de rodar um sistema operacional visitante completo para cada aplicação.

A motivação estrutural do **Docker** foi destruir esse gargalo. Ele padroniza a entrega empacotando o código, o *runtime* e as ferramentas do sistema em um formato universal, garantindo portabilidade absoluta sem emular hardware.



What is Docker?: https://docs.docker.com/get-started/docker-overview/

1 - Imagens vs. Containers

Para projetar infraestrutura moderna, a separação estrita de dois conceitos operacionais é inegociável:

- **Imagens:** É o artefato estático e totalmente imutável. Uma imagem contém o sistema base, os binários e o código da aplicação em camadas *read-only*. Ela não processa dados e não consome ciclos de processador. É a planta baixa do sistema que apenas ocupa armazenamento em disco.
- **Containers:** É a imagem instanciada em tempo de execução. É o instante em que a planta baixa ganha vida na memória **RAM**, materializando-se como um processo ativo, em execução e logicamente isolado, consumindo recursos de **CPU**.

2 - Docker Engine

O Docker não é um bloco monolítico. A gestão do ciclo de vida dos processos é feita pelo **Docker Engine**, operando sob uma arquitetura delegada *Client-Server*:

- **Docker CLI:** O cliente de interface de comando. Ele não executa a conteinerização localmente, apenas traduz os comandos do usuário em requisições **HTTP**.
- **REST API:** A interface de comunicação interna que escuta as requisições do cliente e transmite as instruções para o servidor.
- **Docker Daemon (dockerd):** Ele orquestra a logística de alto nível: gerencia o cache local, baixa imagens dos repositórios, provisiona redes e atrela volumes.
- **containerd e runc:** Historicamente, o *Daemon* fazia todo o trabalho. Hoje, para impedir que uma falha no motor derrube a máquina inteira, a execução é descentralizada. O *Daemon* repassa as ordens para o **containerd**. Quem conversa com o kernel do Linux e levanta as barreiras de isolamento do processo em milissegundos é o **runc** (*OCI Runtime*).

Containers vs. Máquinas Virtuais

Para resolver os gargalos de infraestrutura da Web Solutions, é imperativo abandonar o modelo legado de virtualização. Uma Máquina Virtual (VM) emula hardware do zero e exige a execução de um sistema operacional visitante inteiro apenas para rodar uma aplicação.

Isso gera redundância severa: desperdiça-se RAM e ciclos de CPU apenas para manter processos básicos de sistema rodando em *background* , reduzindo a capacidade real do servidor.

A solução adotada utiliza o **Docker**. O Docker resolve essa ineficiência eliminando a necessidade de emular o hardware. Ele isola as aplicações aproveitando o Kernel do próprio sistema operacional base.

Não há redundância. O isolamento lógico entre os serviços é mantido, mas o overhead é reduzido.

A ilusão de que o container é uma máquina separada acontece pela combinação direta de três recursos primitivos do kernel Linux:

- **chroot:** O *Change Root* pega o processo, o joga dentro de um diretório específico da máquina e altera a sua percepção de espaço. Para o container, aquela pasta minúscula é o diretório raiz (/) do sistema inteiro. Ele está fisicamente cego e preso, impossibilitado de enxergar os arquivos reais do *host*.

- **Namespaces:** Fatiam o que os processos conseguem ver no kernel. O Docker aplica *Namespaces* para isolar IDs de processos, Redes e Comunicação. O seu código sobe e acha que é o PID 1 (o principal processo do sistema). Ele olha para as interfaces de rede e vê uma eth0 isolada , sem ter a menor ideia de que divide o processador físico com outros processos.

- **cgroups:** Realizam a gestão ditatorial dos recursos do hardware. É através dos *Control Groups* que limitamos fisicamente o consumo dos containers. Se o processo tentar ultrapassar a cota de memória designada, o kernel do Linux age: o OOM Killer (*Out Of Memory*) é chamado e mata o processo instantaneamente, mantendo o servidor hospedeiro intacto.

A Importância do YAML

Todo container criado com docker run é imperativo: você digita o comando, a máquina obedece na hora, e se o processo morrer, ninguém vem religar sozinho. Isso funciona para teste rápido. Não funciona para produção.

Kubernetes resolve esse problema virando o jogo do imperativo pro declarativo: você não diz *como* fazer, você diz *o que* quer que exista, e o cluster se vira pra manter aquele estado vivo o tempo todo.

É exatamente isso que o k8s-deploy.yaml do projeto faz. Ele não é um script de instruções sequenciais, é uma declaração de estado desejado que o Kubernetes lê, interpreta e persegue continuamente através do que se chama de **control loop** (reconciliation loop) que veremos em breve.

Por que declarativo e não imperativo?

- **Estado desejado vs estado atual:** você declara "quero 1 réplica do Nginx rodando" e o *kube-controller-manager* compara isso com a realidade do cluster a cada poucos segundos.

- **Auto-cura automática:** se o pod do Nginx cair, o controle loop detecta a divergência entre o declarado e o real, e recria o pod sem intervenção humana, foi por isso que o Deployment do arquivo existe, não um Pod solto.

- **Idempotência:** rodar kubectl apply -f k8s-deploy.yaml dez vezes seguidas produz sempre o mesmo resultado. Rodar um script imperativo dez vezes pode empilhar erro em cima de erro.

Por que YAML especificamente?

YAML venceu como formato porque é legível por humano e estruturado o suficiente para a máquina, mas isso é só a superfície. O que importa de verdade é que o YAML aqui não é configuração solta, ele é a interface de comunicação direta com a API do Kubernetes.

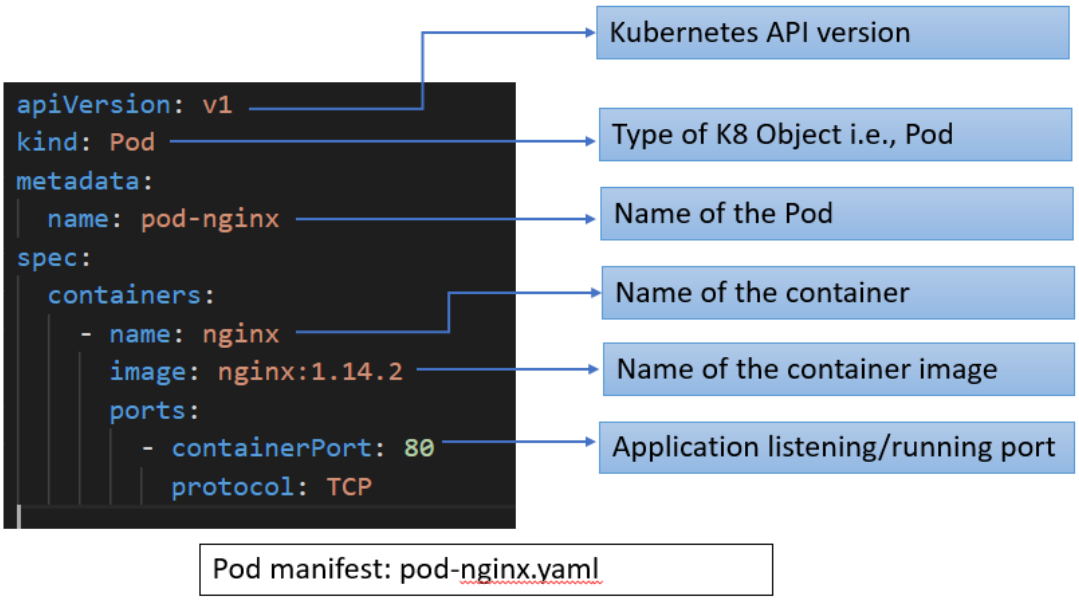
- apiVersion e kind dizem à API qual recurso está sendo declarado (Deployment, Service etc.) e qual versão do schema validar.

- metadata e labels (como app: nginx) não são cosméticos, são o mecanismo de *selector* que o Service usa pra descobrir quais pods deve rotear tráfego. Sem o matchLabels batendo com o labels do template, o Service simplesmente não encontra os pods e a aplicação fica inacessível mesmo com tudo "Running".

- O --- separando os blocos no mesmo arquivo permite declarar múltiplos recursos independentes (Deployment + Service do Nginx, Deployment + Service do Apache) numa única aplicação de manifesto, mantendo a orquestração dos dois servidores coesa num só kubectl apply.

O uso de um arquivo YAML pra declarar os recursos (2 Deployments + 2 Services) reforça a filosofia de Infraestrutura como Código: o arquivo é versionável, auditável e reproduzível em qualquer máquina com Kubernetes ativo, eliminando a dependência de passos manuais decorados ou documentação separada do comportamento real do sistema.

Yaml de exemplo



Pod manifest: pod-nginx.yaml

Fonte: Kubernetes for Developers #10: Kubernetes Pod YAML manifest in-detail

O Pod

O Kubernetes nunca roda containers direto. Ele sempre embrulha um ou mais containers dentro de um Pod e passa a enxergar só esse pacote.

O Pod é a menor unidade que o cluster leva a sério na hora de agendar processos em nós físicos.

Um Pod não é um “container com nome chique”. Ele é um grupo de containers que compartilham o mesmo espaço de rede, o mesmo IP e os mesmos volumes. Do lado de fora, o que existe é o Pod. Os containers lá dentro são detalhe de implementação.

Localhost dentro do Pod

Toda a lógica de Namespaces que já apareceu na comparação entre Containers e VMs volta aqui, só que invertida. Em vez de isolar, o Pod junta.

Os containers do mesmo Pod dividem o mesmo Network Namespace: mesmo IP, mesmas portas.

De dentro do Pod, falar com localhost:80 não é falar com o host físico. É falar com outro container do mesmo Pod que está escutando na porta 80.

Isso tem duas consequências diretas:

- Containers dentro de um Pod conversam por localhost, sem precisar de Service intermediário.
- Dois containers no mesmo Pod não podem escutar a mesma porta ao mesmo tempo, senão brigam pelo mesmo namespace de rede, exatamente como dois processos normais tentando pegar a porta 80 no mesmo Linux.

No projeto, cada Pod tem só um container: ou o Nginx, ou o Apache. Por isso toda essa dinâmica fica invisível.

Mas a arquitetura já está pronta pra um sidecar amanhã, por exemplo, um container auxiliar só pra exportar métricas ou logs, e ele entraria no mesmo Pod, dividindo IP e volumes com o container principal.

O que o scheduler realmente agenda?

Quando o kube-scheduler decide onde rodar alguma coisa, ele não olha pra container individual. Ele só trabalha com Pods. O que cai em cima de um nó é sempre o Pod completo, com todos os containers que fazem parte dele.

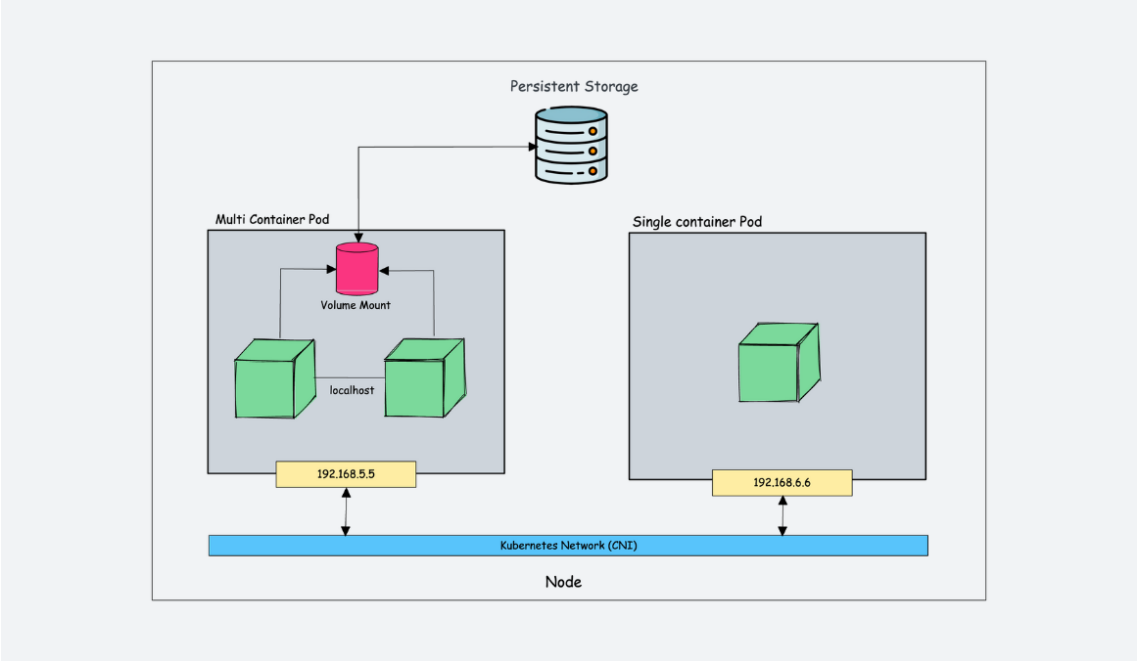
Isso reescreve como ler resources: os requests e limits de todos os containers do Pod somam e viram o “peso” daquele Pod na hora do agendamento. Num Pod de um container só, essa soma é trivial. Num Pod com sidecar, não é mais, o scheduler sempre enxerga o pacote inteiro.

Por que o Pod existe?

Deployments falam de “quantas cópias” você quer rodando. Services falam de “como o tráfego chega até elas”. Nenhum dos dois é uma coisa concreta que ocupa IP e consome CPU. Essa coisa concreta é o Pod.

É ele que nasce, morre, recebe um IP único, entra e sai da reconciliação, e é ele que o Kubernetes move de nó em nó quando precisa. Sem essa camada, containers não teriam uma identidade comum nem uma unidade de ciclo de vida pra nascer e morrer juntos.

É o Deployment que assume a responsabilidade de garantir quantas cópias desse Pod precisam existir, e o que acontece quando uma delas desaparece.



Fonte: Pods

A Anatomia do Deployment

O Deployment é a peça central do k8s-deploy.yaml, mas ele não gerencia pods diretamente, delega isso a um objeto interno chamado ReplicaSet, responsável por garantir que o número de réplicas declarado sempre bata com o número real de pods rodando.

Quando replicas: 1 é definido no manifesto do Nginx, o Kubernetes não apenas sobe um container e esquece o assunto: ele mantém um controle ativo e permanente sobre aquele estado.

Esse comportamento é o que se chama de **reconciliation loop**, ou loop de reconciliação, o mecanismo que sustenta o self-healing dos containers.

Na prática, funciona em quatro etapas contínuas:

- **Observação do estado desejado**: o controller lê o *spec.replicas* declarado no YAML e sabe quantos pods deveriam existir.
- **Observação do estado real**: consulta a API do Kubernetes para saber quantos pods estão de fato rodando naquele momento.
- **Comparação**: se houver divergência entre os dois estados, por exemplo, um pod do Apache crashou e agora existem 0 em vez de 1, o controller identifica o gap.
- **Correção automática**: um novo pod idêntico é agendado e criado, seguindo exatamente a mesma especificação do template original (mesma imagem, mesmas portas, mesmos limites de CPU e memória), fechando a diferença sem qualquer intervenção manual.

Esse loop roda indefinidamente, não em um único disparo. Enquanto o Deployment existir, o Kubernetes vai tentar fechar qualquer divergência entre estado desejado e estado real, seja ela causada por falha de container, reinício de nó ou até exclusão manual acidental de um pod.

Reconciliation Loop

O erro mais comum ao explicar esse mecanismo é descrevê-lo como um script que "fica checando de tempos em tempos". O controller do Deployment se inscreve na API do Kubernetes via um mecanismo chamado **watch**, recebendo eventos assim que qualquer mudança acontece no cluster: criação, atualização ou remoção de um recurso.

Só que aqui está o detalhe técnico que faz toda a diferença: o evento dispara o reconcile, mas o reconcile em si **não reage ao evento**. Ele ignora completamente o que mudou e faz uma pergunta única, sempre a mesma: "o estado atual bate com o estado desejado, sim ou não?".

Level-triggered vs edge-triggered

Esses dois termos são herança de projetos de circuitos elétricos, e a diferença é o que sustenta a resiliência inteira do Kubernetes.

- **Edge-triggered:** reage à *transição* de um evento. Se o sistema perder aquele evento específico, por instabilidade de rede, reinício do controller, o que for, a ação correspondente nunca acontece. Perdeu o evento, perdeu a correção, permanentemente.

- **Level-triggered:** reage ao *estado* em si, não à mudança que levou até ele. Se o controller perder um evento, não importa, na próxima leitura de estado ele vai perceber a divergência de qualquer forma e corrigir. Nenhum evento é indispensável, porque a verdade nunca está no evento, está no estado atual.

O Kubernetes é level-triggered por decisão de design, não por acidente. É isso que permite ao Deployment se recuperar de qualquer falha, mesmo daquelas que ele nunca "viu" acontecer diretamente.

Por que isso importa?

Se o pod do Apache morrer enquanto o controller estava momentaneamente fora do ar (reiniciando, por exemplo), um sistema edge-triggered perderia aquela falha pra sempre, o pod simplesmente ficaria morto.

No modelo level-triggered do Kubernetes, assim que o controller volta, ele lê o estado atual do zero, vê que 0 pods rodando ≠ 1 pod desejado, e corrige na hora, sem depender de ter "visto" o crash acontecer. Resiliência de graça, sem precisar programar retry manual para cada tipo de falha possível.

Rolling Updates como efeito colateral

Esse mesmo mecanismo de reconciliação sustenta os rolling updates: ao alterar a tag da imagem no YAML (de meu-nginx:v1 para meu-nginx:v2, por exemplo) e reaplicar o manifesto, o Deployment não substitui o pod antigo abruptamente. Ele sobe o novo pod, aguarda sua disponibilidade, e só então desliga o anterior, garantindo continuidade de serviço sem downtime perceptível.

Fontes: Kubernetes Docs – Reconciliation Loop, Deployment, A Deep Dive Into The Internals

Resources e imagePullPolicy

Resources não é boilerplate de YAML. É o cgroup do container vazando pra dentro do manifesto declarativo, a mesma engenharia que isola CPU e memória no Docker, só que agora o Kubernetes decide isso antes mesmo do container nascer.

Requests decide onde e Limits decide quanto.

Tem gente que trata requests e limits como sinônimo de "reserva de recurso". Não são. Cada um resolve um problema em momento diferente do ciclo de vida do pod.

requests é usado uma única vez, na hora do agendamento. O *kube-scheduler* olha pra 64Mi e 100m declarados no YAML e só coloca o pod num nó que tenha essa sobra disponível. Depois disso o request não importa mais, ele já cumpriu o papel dele.

limits age depois, em runtime, direto no kernel. 128Mi de memória não é sugestão: é o teto que o cgroup impõe fisicamente. Passou disso, o OOM Killer mata o processo na hora, sem pedir licença. É o mesmo mecanismo que já apareceu na comparação Container vs VM, só que agora configurado via campo do YAML em vez de flag manual do docker run.

O scheduler nunca olha pro limit. O kernel nunca olha pro request. São dois controles cegos um pro outro, cada um vivendo numa camada diferente do sistema.

Por que isso não é opcional?

Sem esses dois valores, um container com memory leak simplesmente consome tudo que o nó físico tem, e derruba os outros containers que estão dividindo aquela máquina junto. Não é teoria: é o "vizinho barulhento" clássico de qualquer ambiente compartilhado, e é exatamente o problema que a arquitetura de containers promete resolver desde a raiz. resources transforma essa promessa em imposição de kernel, não em confiança.

Never existe porque a imagem nunca saiu do disco local

O Kubernetes tem três políticas de busca de imagem: Always, IfNotPresent, Never. Cada uma responde uma pergunta diferente sobre de onde a imagem vem.

Always vai atrás do registry remoto toda vez que sobe um container, faz sentido pra tag mutável tipo :latest.

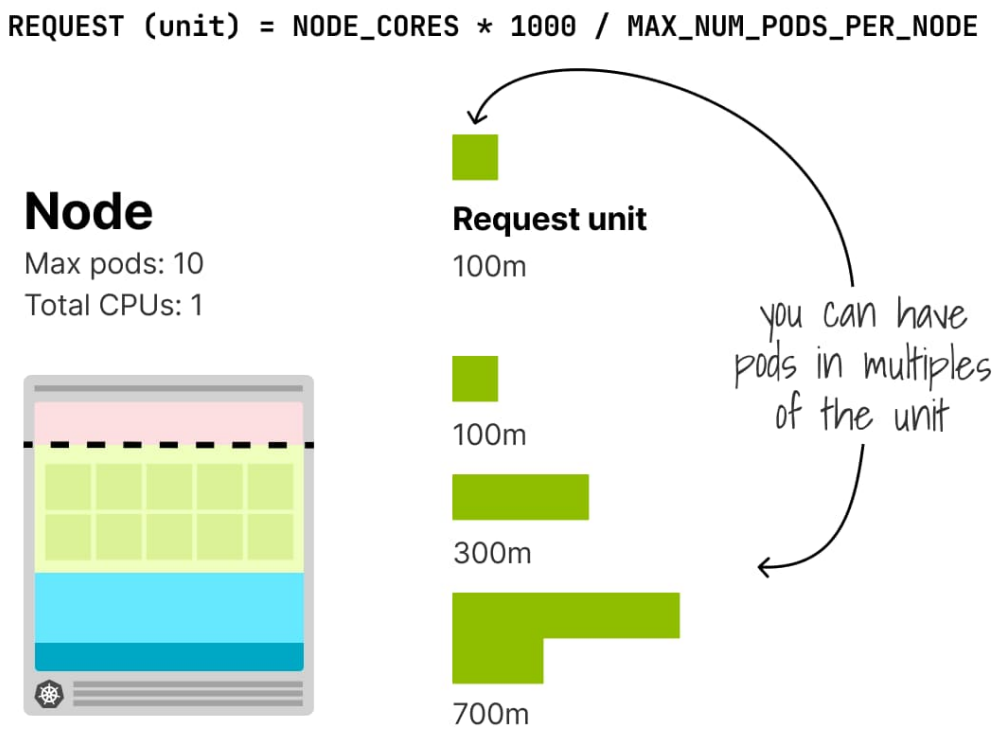
IfNotPresent só busca remoto se a imagem ainda não existir no nó, é o padrão de produção com tag fixa.

Never não busca nada, nunca. Se a imagem não estiver ali, o pod quebra.

No projeto, meu-nginx:v1 e meu-apache:v1 nasceram do docker build local e nunca foram publicadas em nenhum registry, nem no Docker Hub, nem nenhum outro.

Declarar Always ou IfNotPresent faria o Kubernetes tentar puxar essas imagens da internet, não encontrar em lugar nenhum, e estourar ImagePullBackOff.

Never é o único valor coerente com uma imagem que só existe no disco da própria máquina de desenvolvimento, em produção real, o fluxo muda: a imagem vai pra um registry, e Never deixa de fazer sentido.



Fontes: [Kubernetes limits vs request](), [Kubernetes limits & requests]()

Services

Todo pod que você já viu subir tem um problema: o endereço IP dele é descartável. O pod morre no reconciliation loop, o Deployment recria outro, e o IP muda sempre. Se o frontend guardasse esse IP na mão, quebraria a cada reconciliação.

O Service resolve isso criando um endereço estável que nunca muda, na frente de um conjunto de pods que muda o tempo todo. Ele não sabe nada sobre containers, imagens ou build, ele só sabe apontar tráfego pra quem casa com o selector declarado.

É por isso que app: nginx no Service e app: nginx no matchLabels do Deployment precisam ser idênticos: é o único fio que liga os dois objetos.

Não existe o "Service genérico". Existem três tipos empilhados um em cima do outro, cada um ampliando o alcance do anterior:

- **ClusterIP:** o padrão, se você não declarar type. Cria um IP interno só visível de dentro do cluster, bom pra comunicação pod-a-pod, inútil se você quer acessar do seu navegador.

- **NodePort:** pega o ClusterIP de base e abre uma porta fixa (entre 30000-32767) em todo nó físico do cluster. Já dá pra acessar de fora, mas com uma porta feia e alta, sem elegância nenhuma.

- **LoadBalancer:** pega o NodePort de base e coloca um balanceador de carga externo na frente dele. É o único que permite mapear para uma porta arbitrária que você escolhe, no caso do projeto, 8080 pro Nginx e 8081 pro Apache.

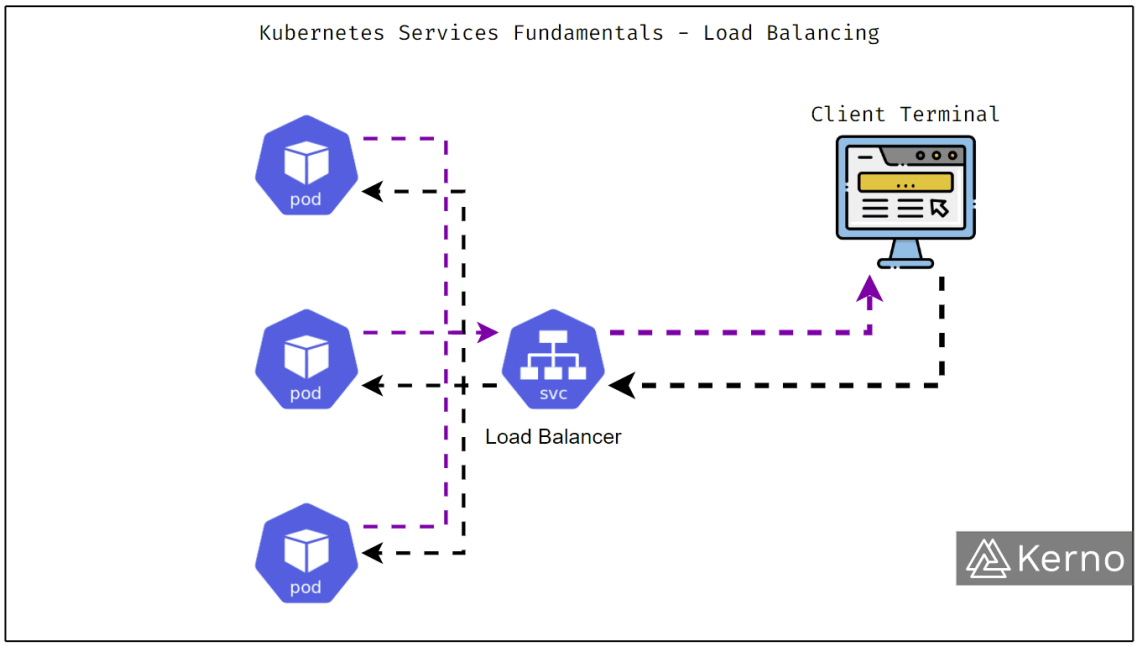
Cada tipo não substitui o anterior, ele o envolve. LoadBalancer sem NodePort por baixo simplesmente não teria como rotear o tráfego externo até dentro do cluster.

Por que o projeto usa LoadBalancer?

Em produção real, LoadBalancer dispara a criação de um balanceador na AWS, GCP ou Azure, o cloud provider é quem realmente cria a máquina que recebe o tráfego externo. Localmente, no Docker Desktop, não existe cloud provider nenhum: o próprio Docker Desktop simula esse comportamento e mapeia a porta declarada (8080, 8081) direto pro localhost da sua máquina. É por isso que abrir o navegador em localhost:8080 funciona sem nenhuma configuração de nuvem, o ambiente local finge ser um provedor de nuvem só o suficiente pra fazer o type: LoadBalancer funcionar de verdade.

port vs targetPort

O YAML do projeto declara port: 8080 e targetPort: 80 no Service do Nginx. Não é redundância, são dois lados diferentes da mesma ponte: targetPort é a porta que o container escuta por dentro (80, porque é onde o Nginx roda nativamente), e port é a porta que o Service expõe pra fora, pro cliente que está chamando. O Service faz essa tradução de porta sem o container saber que ela existe.



Fonte: Services

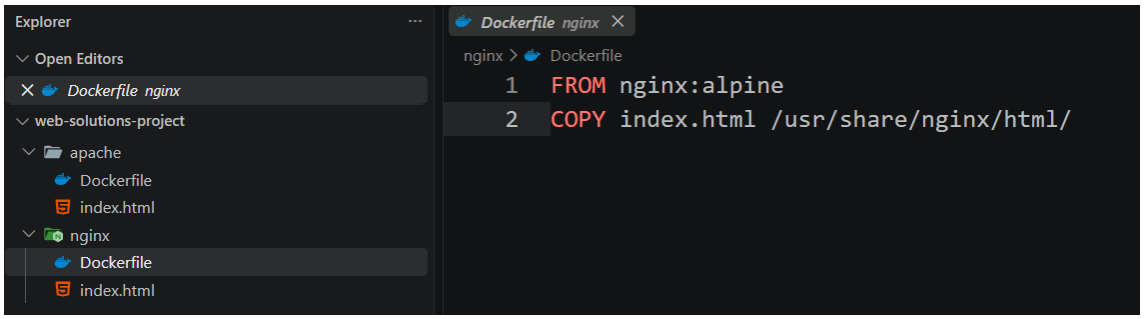
A Prova de Conceito da Web Solutions

O ponto de partida era simples: dois servidores web diferentes, Nginx e Apache, rodando no mesmo cluster, em portas separadas, sem brigar por recurso e sem depender de máquina virtual dedicada pra cada um.

Fase 0 - *Ambiente*

Antes de qualquer container existir, o ambiente local precisava virar um cluster. Docker Desktop resolve isso sem-cerimônia: instala o motor de containers e vem com um cluster Kubernetes embutido, pronto pra ativar nas configurações. Basta marcar "Enable Kubernetes", aplicar, aguardar o cluster inicializar, e a máquina passa a ter um nó Kubernetes funcional rodando localmente.

A raiz do projeto ficou numa pasta chamada web-solutions-project. Tudo que compõe a infraestrutura: Dockerfiles, páginas HTML, manifesto YAML, moram dentro dela. Nenhuma configuração existe fora do repositório.



Fase 1 - *As Imagens*

Em vez de puxar uma imagem genérica do Docker Hub e servir o conteúdo padrão, Nginx e Apache foram empacotados com identidade própria.

Dois motivos pra isso:

Primeiro: Na hora de testar, precisa ser óbvio qual servidor está respondendo.

Segundo: Imagem customizada significa que você controla o que entra nela, não tem surpresa de atualização silenciosa quebrando o ambiente.

A estrutura ficou assim: duas subpastas, nginx e apache, cada uma com seu próprio index.html e seu Dockerfile.

A estilização dos arquivos index.html em formato de console simula um ambiente de produção real.

Os Dockerfiles são mínimos por decisão, não por preguiça:

```
# nginx/Dockerfile
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/

# apache/Dockerfile
FROM httpd:alpine
COPY index.html /usr/local/apache2/htdocs/
```

A base Alpine não é detalhe estético. A imagem nginx:alpine pesa menos de 25MB contra os ~140MB da versão Debian.

Menos pacote instalado significa menos superfície de ataque e menos camada pra manter atualizada.

Em ambiente de produção isso importa. Aqui, mesmo sendo uma PoC local, a decisão foi tomada como se importasse, porque eventualmente vai importar.

Antes de prosseguir quero falar sobre Layers.

Cada linha de um Dockerfile que modifica o sistema de arquivos gera uma camada nova empilhada em cima da anterior. Não é uma imagem monolítica, é uma pilha de diffs binários, funcionando de forma muito parecida com commits no Git.

Quando o docker build executa o Dockerfile do Nginx:

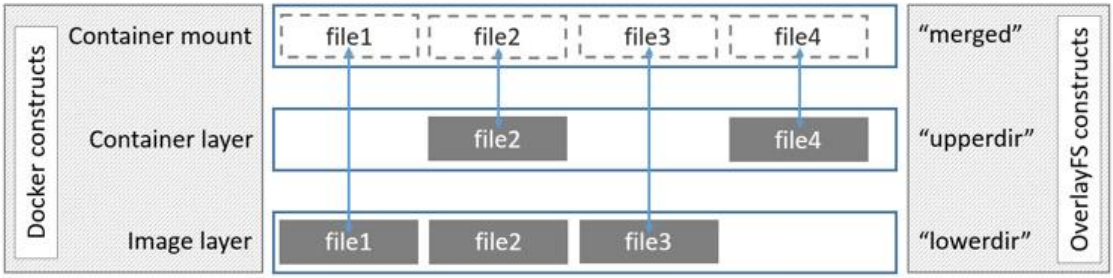
```
FROM nginx:alpine       # camada base - já existe no cache do Docker Hub
COPY index.html /usr/share/nginx/html/  # nova camada - só o arquivo copiado
```

A imagem meu-nginx:v1 não tem 25MB duplicados da base Alpine mais o index.html. Ela tem a camada base do nginx:alpine referenciada, e uma segunda camada minúscula contendo apenas o index.html que foi copiado.

Se amanhã trocarmos só o index.html e rodarmos o build de novo, o Docker reusa a camada base do cache e só reconstrói a última, o que torna rebuilds rápidos por definição.

Isso é Copy-on-Write: em vez de duplicar o que não mudou, cada camada armazena apenas o diff do que foi alterado em relação à anterior. É o mesmo conceito do QCOW2 que sistemas de virtualização usam pra snapshots de disco, só que aplicado a imagens de container.

Para montar o sistema de arquivos que o container vai enxergar em tempo de execução, o Docker usa o **overlay2**, um Union Filesystem que pega todas as camadas e as unifica numa visão única e coerente.



A imagem acima mostra o overlay2 em ação. A lowerdir é a camada read-only, no projeto, é o nginx:alpine com todos os arquivos base do servidor.

A upperdir é onde o container escreve durante a execução: modificações e arquivos novos aparecem aqui, nunca na camada base. O merged é o que o container realmente enxerga, uma visão unificada das duas camadas, como se fosse um sistema de arquivos normal.

Quando o container do Nginx é destruído, a upperdir some junto. O nginx:alpine na lowerdir permanece intacto, pronto pra ser reutilizado no próximo docker run sem reconstruir nada.

O container não vê camadas separadas. Ele vê um sistema de arquivos normal, como qualquer outro Linux. As camadas são detalhe de construção, não de execução.

Uma consequência direta disso é como você deve escrever o Dockerfile. Se o rm de arquivos temporários ficar num RUN separado do apt-get install que os gerou, a camada com os arquivos inchados já foi gravada o rm cria uma camada que os esconde, mas não os apaga da imagem. O tamanho final da imagem carrega os dois. Por isso o padrão correto é encadear tudo no mesmo RUN:

```
RUN apt-get update && apt-get install -y pacote && rm -rf /var/lib/apt/lists/*
```

No projeto, os Dockerfiles são simples o suficiente pra não ter esse problema, FROM alpine já garante base enxuta e o único COPY não gera resíduo. Mas entender esse mecanismo é o que separa quem escreve um Dockerfile que funciona de quem escreve um que funciona e pesa o mínimo necessário.

Com os Dockerfiles prontos, as imagens foram construídas:

```
docker build -t meu-nginx:v1 ./nginx
docker build -t meu-apache:v1 ./apache
```

A partir daqui o projeto parou de depender da internet pra ter Nginx e Apache. As duas imagens existem no cache local, com tag fixa v1, prontas pro Kubernetes usar sem chamar nenhum registry externo.

Fonte: Dockerfile reference, Layers

Fase 2 - *O Manifesto*

Com as imagens prontas, o próximo passo foi materializar a infraestrutura em código. Um único arquivo, k8s-deploy.yaml, na raiz do projeto, concentra os quatro recursos que colocam a PoC de pé: dois Deployments e dois Services, um par pra cada servidor.

Manter tudo num arquivo só não é preguiça, é decisão.

Um único kubectl apply -f k8s-deploy.yaml aplica Nginx e Apache juntos, na ordem certa, sem depender de scripts auxiliares ou passos manuais memorizados. O estado inteiro do cluster está legível, versionável e reproduzível por qualquer pessoa que clone o repositório.

O --- separando os blocos dentro do arquivo é o que torna isso possível: cada bloco declara um recurso independente, mas o Kubernetes os processa como uma unidade coesa na mesma operação.



Fonte: Deployments

Fase 3 - *Execução e Validação*

Com o ambiente pronto, as imagens construídas e o manifesto declarado, uma linha resolve o resto:

kubectl apply -f k8s-deploy.yaml

O Kubernetes leu o arquivo, comparou com o estado atual do cluster e criou os quatro recursos de uma vez. Nenhuma ordem manual, nenhum script de inicialização, nenhum passo fora do repositório.

Para confirmar que tudo subiu:

kubectl get pods

```
PS C:\Users\deton\OneDrive\Areas\Faculdade\Pedro\Container e kubernets\web-solutions-project> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
apache-deployment-b465c45d5-75mbd   1/1     Running   0          20s
nginx-deployment-75b4674d9-s4wvj    1/1     Running   0          20s
PS C:\Users\deton\OneDrive\Areas\Faculdade\Pedro\Container e kubernets\web-solutions-project>
```

STATUS: Running e READY: 1/1 significam que o reconciliation loop fechou o estado desejado e o cluster está estável.

Qualquer coisa diferente disso, Pending, CrashLoopBackOff ou ImagePullBackOff, é divergência entre o que foi declarado e o que o cluster conseguiu provisionar, e o lugar certo pra começar a debugar é exatamente ali.

Com os pods no ar, a validação foi direto no navegador:



Dois servidores distintos, portas separadas, mesmo cluster, sem conflito de recurso.

Fonte: kubectl apply

Conclusão

A proposta era simples: dois servidores web diferentes, mesmo cluster, sem VM, sem configuração manual, sem torcer pra não quebrar.

O que fica de aprendizado não é a lista de comandos. É entender por que cada camada existe e o que ela resolve de verdade. Docker não é "container mágico", é chroot mais namespaces mais cgroups com uma boa camada de tooling por cima.

A imagem não é um executável monolítico, é uma pilha de layers Copy-on-Write onde cada linha do Dockerfile é um diff binário empilhado sobre o anterior.

O overlay2 cola tudo isso numa visão única que o container enxerga como sistema de arquivos normal, sem saber que está olhando pra múltiplas camadas sobrepostas.

Kubernetes não é "orquestrador automático", é um loop de reconciliação rodando indefinidamente, comparando o que foi declarado com o que existe de verdade no cluster, e corrigindo a diferença.

Esse loop é level-triggered por design: não reage a eventos, reage a estado. Se o controller perder um evento de falha, não importa, na próxima leitura ele enxerga a divergência e corrige de qualquer forma.

É isso que torna o self-healing confiável de verdade, não uma promessa de marketing.

O Pod define a unidade real que o scheduler enxerga.

O Deployment garante quantas cópias dessa unidade precisam existir.

O Service abstrai o endereço instável dos pods e entrega um ponto fixo de acesso.

O YAML une tudo isso num artefato único, declarativo, versionável e reproduzível em qualquer cluster com dois comandos.

Quando o pod do Apache morrer às 3 da manhã, ninguém vai acordar pra reiniciar. O cluster já corrigiu antes de qualquer alerta disparar. Isso não é feature de Kubernetes, é consequência direta de entender o que está sendo declarado no manifesto.

A PoC rodou num nó local, réplica única, imagens de poucos megabytes. A estrutura é idêntica à de um cluster de produção com dezenas de nós e centenas de pods. O que muda é a escala. O raciocínio é o mesmo.

Fonte: Kubernetes Overview